Odd and Even Game
홀짝게임

-libpng 기반 이미지 처리 프로그램을 중심으로-

영원 조

20231508 강유영

20231528 김소원

목차

제 1장 개발 환경

제 2장 프로젝트 설계
2.1 게임 알고리즘
2.2 화면 출력

제 3장 프로젝트 구현
3.1 게임 알고리즘 코드
3.2 구슬 출력
3.3 WIN, LOSE 출력
3.4 output.png 갱신

제 4장 컴파일 및 결과 출력
4.1 maketool을 이용한 컴파일
4.2 결과 출력

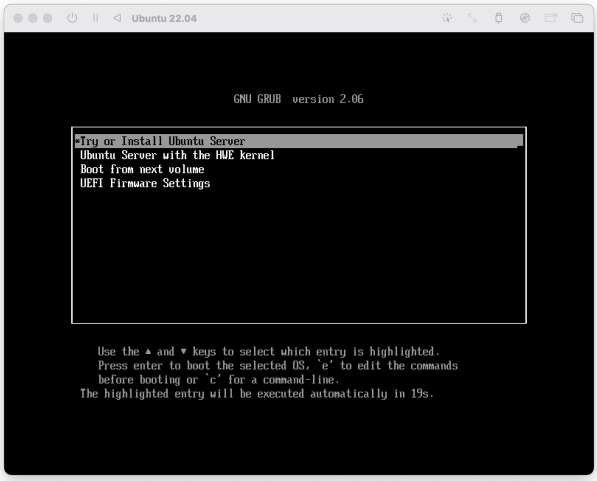
제 5장 결론

*참고문헌

제 1장 개발 환경

이 코드는 mac 운영체제에서 UTM을 통해 리눅스에 접근하여 우분투를 다운로드하고, 우분투와 vscode의 원격 서버 연결을 통해 작성되었다. 언어는 c++을 사용하였다. 아래는 mac 운영체제에서 우분투를 다운로드해 vscode와 연결하는 과정이다.

1) 우분투 파일을 다운로드하고, UTM에 우분투를 설치한다.

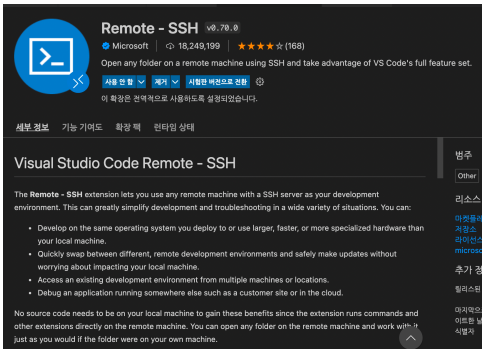


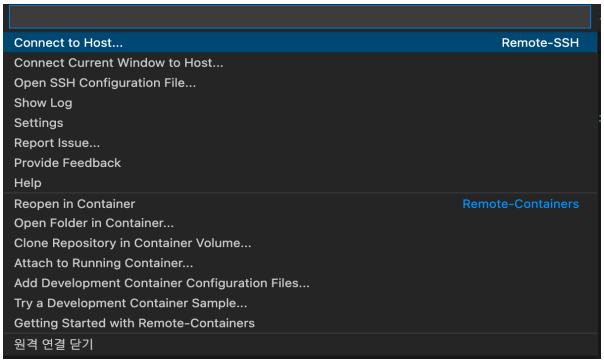
2) sudo apt -y install gcc 명령어를 통해 우분투에 g++를 설치한다.

3) sudo apt-get install libpng-dev, sudo apt-get install zlib1g-dev 명령어를 통해 libpng를 설치한다.

4) mkdir, cd를 이용해 파일을 생성한다.

5) 기존의 vscode에서 Remote-SSH 확장 프로그램을 설치한다. Connect to Host를 통해 우분투와 vscode가 원격으로 연결될 수 있게 한다. 원격 연결을 하면 우분투에서 mkdir을 이용해 만든 파일을 확인할 수 있다.





제 2장 프로젝트 설계

2.1 게임 알고리즘

플레이어와 컴퓨터가 각각 6개의 구슬을 가지고 시작한다. 서로가 내건 구슬의 수가 짝수일지 홀수일지를 맞추며 승리한 쪽이 구슬을 가져가고, 한 쪽이 구슬을 모두 얻게 되면 끝난다.

컴퓨터가 베팅을 하고 플레이어가 홀, 짝을 입력하는 경우 플레이어가 정답을 맞히면 컴퓨터가 베팅한 구슬의 수만큼 플레이어가 가져온다. 플레이어가 정답을 맞히지 못하면 컴퓨터가 베팅한 구슬의 수만큼 플레이어가 구슬을 뺏긴다.

플레이어가 베팅을 하고 컴퓨터가 홀, 짝을 출력하는 경우 컴퓨터가 정답을 맞히면 플레이어가 베팅한 구슬의 수만큼 구슬을 뺏긴다. 컴퓨터가 정답을 맞히지 못하면 플레이어가 베팅한 구슬의 수만큼 플레이어가 구슬을 가져온다.

위의 과정을 계속 반복한 후 플레이어 구슬의 수가 12 이상이 되면 WIN 화면을 출력한다. 구슬의 수가 0 이하가 되면 LOSE 화면을 출력한다. 구슬의 수가 12 '이상', 0 '이하'가 될 때 승패가 결정되게 한 이유는 가지고 있던 구슬의 수보다 더 많은 구슬을 뺏겨야 하는 경우 음수가 출력되기도 하기 때문이다. 예를 들어 3개의 구슬을 가진 상태에서 컴퓨터가 6개의 구슬을 걸고 플레이어가 졌다면 3-6=-3로 플레이어의 현재 구슬 수가 음수가 된다.

2.2 화면 출력

플레이어 입력의 결과에 따라 png 파일이 계속 생기면 비효율적일 것이라 생각해 게임 실행 시 output.png 파일을 만들고 입력에 따른 결과로 이 파일을 계속 갱신할 수 있도록 하였다.

플레이어가 현재 가지고 있는 구슬의 구슬을 시각적으로 쉽게 확인하기 위해 현재 구슬을 화면에 출력한다. updateImage 함수에서 플레이어의 현재 구슬 수가 12 이상, 0이하가 아닌 경우에 플레이어의 현재 구슬 수에 따른 구슬 화면을 출력한다.

화면에 WIN과 LOSE를 출력하기 위해 선을 그리는 함수인 drawLine을 따로 만들어서 사용하였다. updateImage 함수에서 플레이어의 현재 구슬 수가 12개 이상일 경우 drawWin 함수를 실행하고, 0개 이하일 경우 drawLose 함수를 실행한다.

제 3장 프로젝트 구현

3.1 게임 알고리즘 코드

```
int playerbeads = 6;
int computerbeads = 6;
```

플레이어의 구슬 수와 컴퓨터의 구슬 수는 각각 6으로 시작한다.

```
while (playerbeads > 0 && playerbeads < 12) {...}
```

playerbeads가 0부터 12 사이에 있을 때 알고리즘은 계속 반복된다.

```
updateImage("output.png", playerbeads);
```

게임 실행 시 out.png파일이 만들어진다.

게임 시작 시에는 구슬이 6개여서 구슬 6개가 화면에 출력된다.

(updateImage 함수에 대해서는 3.2 구슬 출력에서 다룬다.)

```
int computernumber = rand() % computerbeads + 1;
bool is_c_odd = (computernumber % 2 == 1);
```

컴퓨터가 건 구슬의 수를 랜덤함수를 통해 출력한다. 구슬의 수가 짝수면 0, 홀수면 1을 is_c_odd변수 안에 저장한다.

```
do {
cout <<"홀수입니까? 짝수입니까? (홀수면 1, 짝수면 2를 입력하시오): ";
cin >>playerchoice;
if (playerchoice != 1 && playerchoice != 2) {
cout <<"1 또는 2를 입력해야합니다. 다시 입력해주세요."<<endl;
 }
 } while (playerchoice != 1 && playerchoice != 2);
```

플레이어에게 홀수 또는 짝수를 입력받는다. 1 또는 2가 아닌 다른 수를 입력받았을 경우 다시 입력을 받아야 한다. 이를 효율적으로 나타내기 위해 while 반복문이 아니라 do-while 반복문을 사용하였다.

```
if ((playerchoice % 2 == 1 && is_c_odd) || (playerchoice % 2 == 0 && !is_c_odd))
{
playerbeads += computernumber;
computerbeads -= computernumber;
if (playerbeads >= 12) {
cout <<"당신은 모든 구슬을 얻었습니다!"<<endl;
updateImage("output.png", playerbeads);
break;
 }
else {cout <<"이겼습니다!(현재 구슬 수: "<<playerbeads <<") "<<endl<<endl;
updateImage("output.png", playerbeads);
 }
```

is_c_odd 가 참이고 플레이어가 1을 입력했거나 is_c_odd가 거짓이고 플레이어가 2를 입력했다면 플레이어의 승리이므로 플레이어는 컴퓨터가 건 구슬의 수만큼을 얻고, 컴퓨터는 그만큼 잃는다.

만약 playerbeads가 12이상이 된다면 게임이 끝나므로 결과 화면을 출력하고 게임을

끝낸다. 12이상이 아닌 경우에는 남은 구슬 수만큼 out.png를 갱신한다.
플레이어가 패배한 경우도 위와 같은 알고리즘으로 작동한다.

```
do {
cout <<"구슬 몇 개를 걸겠습니까? (현재 구슬 수: "<<playerbeads <<"): ";
cin >>playerbet;
if (playerbet > playerbeads || playerbet <=0) {
cout <<"구슬 수가 잘못되었습니다. 다시 입력해주세요."<<endl;
 }
 } while (playerbet > playerbeads || playerbet <=0);
```

컴퓨터가 걸고 플레이어가 맞히는 것이 끝났다면 이번에는 플레이어가 걸고 컴퓨터가 맞히어야 할 순서이다. 플레이어에게 구슬을 얼마나 걸지 입력받는다. 만약 입력받은 수가 0이하이거나 playerbeads보다 크다면 옳지 않은 입력이기 때문에 do-while을 통해 다시 입력받는다.

```
int computernumber_2 = rand() % 2;
```

랜덤함수를 통해 컴퓨터가 출력하는 짝 또는 홀을 변수에 저장한다.

```
if ((playerbet % 2 == computernumber_2)) {
playerbeads -= playerbet;
computerbeads += playerbet;
if (playerbeads <= 0) {
cout <<"당신은 모든 구슬을 잃었습니다!"<<endl;
updateImage("output.png", playerbeads);
break;
 }
else {cout <<"졌습니다!(현재 구슬 수: "<<playerbeads <<") "<<endl<<endl;
updateImage("output.png", playerbeads);
 }
```

playerbet과 computernumber_2가 모두 짝수 또는 모두 홀수라면 컴퓨터가 플레이어가 건 구슬이 짝수인지 홀수인지 맞힌 것이므로 플레이어의 패배이다. 따라서 플레이어는 건 구슬 수만큼 잃고 컴퓨터는 얻는다. 만약 플레이어의 구슬 수가 0이하가 됐다면 게임이 끝나므로 결과 화면을 출력한다. 0이하가 아닌 경우에는 남은 구슬 수만큼 out.png를 갱신한다.
플레이어가 승리한 경우에도 위와 같은 알고리즘으로 작동한다.

3.2 구슬 출력

- drawCircle 함수

```
void drawCircle(png_bytep *row_pointers, int x, int y, int radius) {
for (int dy = -radius; dy <= radius; dy++) {
for (int dx = -radius; dx <= radius; dx++) {
if (x + dx >= 0 && x + dx < imageWidth && y + dy >= 0 && y + dy < imageHeight) {
if (dx * dx + dy * dy <= radius * radius) {
row_pointers[y + dy][4 * (x + dx)] = 255;
row_pointers[y + dy][4 * (x + dx) + 1] = 255;
row_pointers[y + dy][4 * (x + dx) + 2] = 255;
row_pointers[y + dy][4 * (x + dx) + 3] = 255;
```

```
 }
 }
 }
 }
}
```

현재 픽셀이 이미지 범위 내에 있는지, 원의 방정식을 만족하는지를 확인한 후 원 내부 픽셀을 흰색으로 채우는 함수이다. row_pointers는 이미지 각 행에 대한 포인터 배열이다. x, y는 원의 중심 좌표이다.

```
void drawBeads(png_bytep *row_pointers, int numBeads) {
const int circleRadius = 20;
const int padding = 40;
for (int i = 0; i < numBeads; i++) {
int centerX = padding + i * (2 * circleRadius + padding);
int centerY = imageHeight / 2;
drawCircle(row_pointers, centerX, centerY, circleRadius);
 }
}
```

원을 그리는 함수 drawCircle을 이용해 구슬을 그리는 함수이다. drawCircle이 원만 그렸다면, drawBeads는 구슬'들'을 그린다. numBeads로 그릴 구슬의 수를 입력받는다. 구슬의 반지름은 20, 구슬 간의 간격은 40으로 설정했다.

3.3 WIN, LOSE 출력

WIN 결과화면을 출력하기 위해서는 먼저 선을 그리는 함수에 대해 설명해야 한다.

- drawLine 함수

```
void drawLine(png_bytep *row_pointers, int x1, int y1, int x2, int y2) {
int dx = abs(x2 - x1);
int dy = abs(y2 - y1);
int sx, sy;
if (x1 < x2) {
sx = 1;
 } else {
sx = -1;
 }
if (y1 < y2) {
sy = 1;
 } else {
sy = -1;
 } ...
```

x좌표의 변화량과 y좌표의 변화량을 계산하기 위해 절댓값을 구하는 abs함수를 사용하였다. sx와 sy는 어떻게 이동할지를 저장하는 변수인데 x1이 x2보다 작으면 sx는 1 그렇지 않으면 -1이 된다. y도 마찬가지이다.

```
int err = dx - dy;
```
err변수를 통해 어느 방향으로 이동할지 정한다. 이후 err의 값이 양수면 가로로 이동하고 음수면 세로로 이동한다. 이 변수는 각 픽셀을 그릴 때마다 조절되며 다음 픽셀을 선택하는 데 영향을 준다.

```
while (true) {
if (x1 >= 0 && x1 < imageWidth && y1 >= 0 && y1 < imageHeight) {
row_pointers[y1][4 * x1] = 255;
row_pointers[y1][4 * x1 + 1] = 255;
row_pointers[y1][4 * x1 + 2] = 255;
row_pointers[y1][4 * x1 + 3] = 255;
 }
```
현재좌표 (x1, y1)에 픽셀을 그린다. if문을 이용해 좌표가 이미지 범위를 벗어나면 픽셀을 그리지 않는다. 매 반복마다 x1, y1을 업데이트하고 직선을 그리게 된다.

```
if (x1 == x2 && y1 == y2) {
break;
 }
```
현재 좌표 x1, y1 이 목표 좌표 x2, y2에 도달했을 때 함수를 종료한다.

- drawWin 함수

```
void drawWin(png_bytep *row_pointers, int x, int y, int fontSize) {
 // 'W' 그리기
drawLine(row_pointers, x, y, x + fontSize / 4, y + fontSize);
```
시작점은 (x,y)이고 끝점은 (x+fontSize, y+fontSize)이다.
이렇듯 drawLine을 이용해 시작점과 끝점을 반복적으로 입력해주어 W, I, N 글자를 출력한다.

- drawLose 함수

drawWin과 마찬가지로 drawLine함수를 이용해 L, O, S, E를 출력한다.

3.4 output.png 갱신

플레이어가 입력을 할 때마다 output_1.png, output_2.png ... 가 생기면 메모리 낭비도 발생하고, 폴더가 상당히 복잡해질 것이다. 그래서 플레이어 입력에 따라 출력 png 화면이 바뀔 때마다 기존의 output.png 파일 위에 새로운 이미지를 덧씌우는 함수를 만들었다.

- updateImage 함수

이 함수는 main()에서 플레이어의 입력에 따라 승리, 또는 패배했을 때 (모든 구슬을 얻거나 잃을 때뿐만 아니라 구슬의 일부를 얻거나 잃었을 때도 포함한다) 사용되며 playerbeads 변수 값에 따라 화면에 출력되는 구슬 수를 갱신하고, WIN 또는 LOSE 화면을 출력할 때도 사용된다.

```
png_init_io(png, fp);
```

png구조체에 대한 파일 입출력을 초기화하고, 'png'구조체가 'fp'파일을 사용하게 한다.

```
png_set_IHDR(
png,
info,
imageWidth, imageHeight,
8,
PNG_COLOR_TYPE_RGBA,
PNG_INTERLACE_NONE,
PNG_COMPRESSION_TYPE_DEFAULT,
PNG_FILTER_TYPE_DEFAULT
 );
```

png 이미지의 기본 형식이 저장되어 있다. 이 함수를 통해 이미지의 너비, 높이, 비트, 색깔 타입, 압축 방식, 필터 방식 등을 설정할 수 있다. (필터 방식은 png 이미지 필터링을 통해 이미지 데이터를 처리하여 효율을 높이는 방법을 의미한다. png_set_IHDR에서는 PNG_FILTER_TYPE_DEFAULT 로 기본 필터 방식을 사용하고 있다.)

```
png_write_info(png, info);
```

png_set_IHDR에서 설정한 내용을 바탕으로 png 이미지의 헤더를 파일에 쓴다.

```
png_bytep *row_pointers = (png_bytep *)malloc(sizeof(png_bytep) * imageHeight);
for (int y = 0; y < imageHeight; y++) {
row_pointers[y] = (png_byte *)malloc(png_get_rowbytes(png, info));
 }
```

row_pointers는 png_bytep 타입의 포인터 배열을 나타낸다. 각 포인터는 이미지의 한 행을 가리키며 malloc 함수를 사용하여 이러한 포인터들을 저장할 메모리를 동적으로 할당한다.

malloc(png_get_rowbytes(png, info))는 한 행에 대한 메모리를 동적으로 할당한다. png_get_rowbytes(png, info)는 해당 행의 바이트 수를 반환하는 함수로, 이미지의 너비와 비트에 따라 달라집니다. 따라서 각 행의 크기를 정확히 할당하려면 이 함수를 사용해야 한다.

이렇게 동적으로 할당된 row_pointers 배열은 이후에 이미지의 픽셀 데이터를 저장하는 데 사용된다. 각각의 포인터가 한 행을 가리키고, 해당 행의 각 바이트가 픽셀의 정보를 담는다.

```
for (int y = 0; y < imageHeight; y++) {
for (int x = 0; x < imageWidth * 4; x += 4) {
float gradient = static_cast<float>(y) / (imageHeight - 1);
row_pointers[y][x] = static_cast<int>((1 - gradient) * 0 + gradient * 0);
row_pointers[y][x + 1] = static_cast<int>((1 - gradient) * 0 + gradient * 0);
row_pointers[y][x + 2] = static_cast<int>((1 - gradient) * 128 + gradient *
255);
row_pointers[y][x + 3] = 255;
 }
 }
```

배경을 초기화 한다. 위에서 아래로 갈수록 밝은 색이 되도록 rgb값을 통해 설정했다.

```
if (numBeads >= 12) {
drawWin(row_pointers, imageWidth / 6, imageHeight / 3, imageHeight / 3);
 } else if (numBeads <= 0) {
drawLose(row_pointers, imageWidth / 6, imageHeight / 3, imageHeight / 3);
 } else {
drawBeads(row_pointers, numBeads);
 }
```

numBeads 값에 따라 출력 화면을 다르게 한다. numBeads가 12 이상이면 drawWin을 실행하고, 0이하이면 drawLose를 실행한다. 그 외의 경우 구슬 수에 따라 구슬 이미지를 출력하는 drawBeads를 실행한다.

```
saveImage(filename, imageWidth, imageHeight, row_pointers);
```

png 이미지를 저장하는 함수를 실행하고 메모리와 파일을 닫으며 updateImage함수의 역할은 끝난다. 이 함수는 main()에서 다시 호출될 때마다 같은 작업을 반복한다. saveImage함수에 관한 설명은 아래에 있다.

- saveImage 함수

파일을 열고, png라이브러리, png정보를 초기화 한다. 그 후 png_set_IHDR(...) 로 png 헤더를 설정하고 png_write_info(png, info) 로 png 정보를 쓰고 파일에 반영한다. 이미지데이터를 쓰고 파일을 닫는 기능까지 수행한다. updateImage와 수행하는 일이 거의 비슷하다고 볼 수 있다.

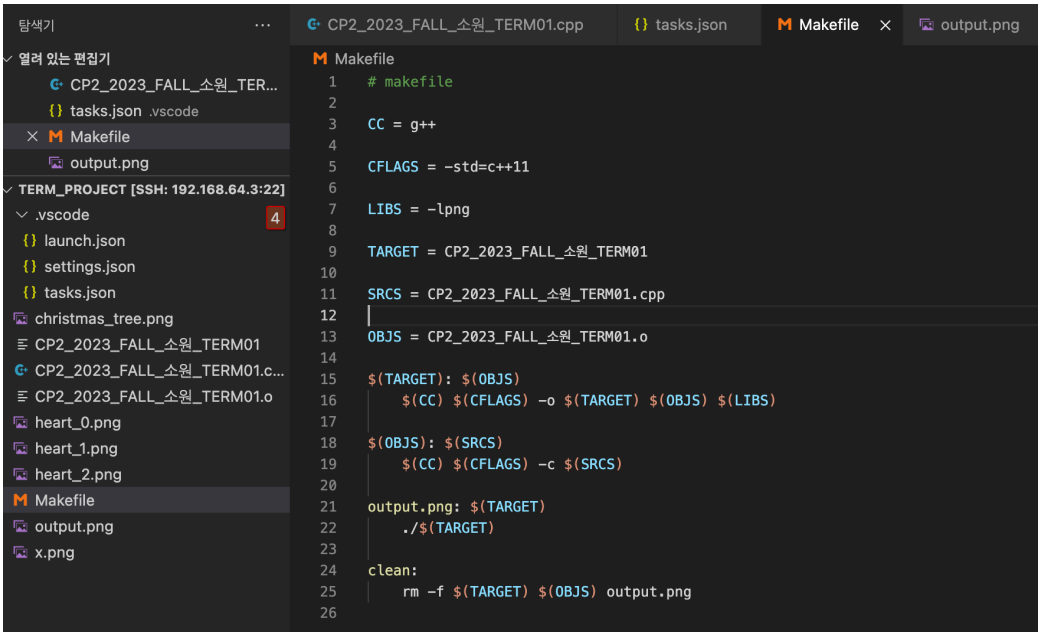
제 4장 컴파일 및 결과 출력

4.1 maketool을 이용한 컴파일

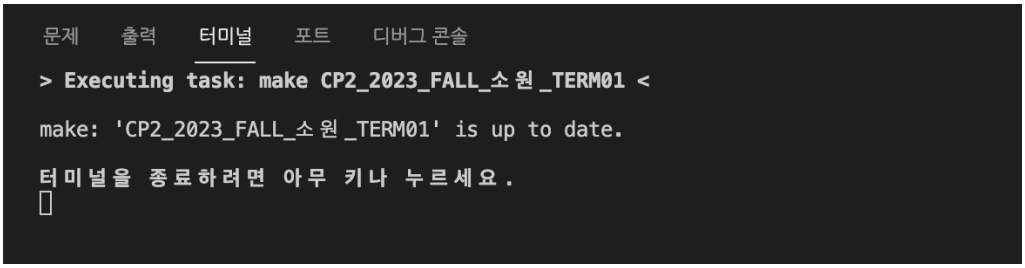
기존 gcc/g++를 이용해서 프로그램 빌드를 할 때 터미널 창에 써야 하는 명령어가 복잡해 힘들 때가 있다. 파일 개수가 적다면 크게 문제 될 것이 없을 수 있지만 프로젝트가 복잡해지면 입력해야 할 명령어가 많아져 여간 번거로운 것이 아니다. make를 이용하면 프로젝트를 더 간단하게 실행할 수 있다.

make는 주어진 터미널 명령어를 조건에 맞게 실행하는 프로그램이라 할 수 있다. 이때 어떠한 조건으로 프로그램을 실행할지를 담은 파일을 Makefile이라고 한다. make를 터미널에서 실행하게 되면 Makefile에 있는 내용을 읽어들인다.

make 명령어를 사용하기 위해 vscode에 Makefile Tools 확장자를 설치했다.
(Makefile Tools 에 작성한 조건은 참고문헌 1을 참고하였다.)



Makefile을 작성했다면 빌드 작업 실행을 한다.



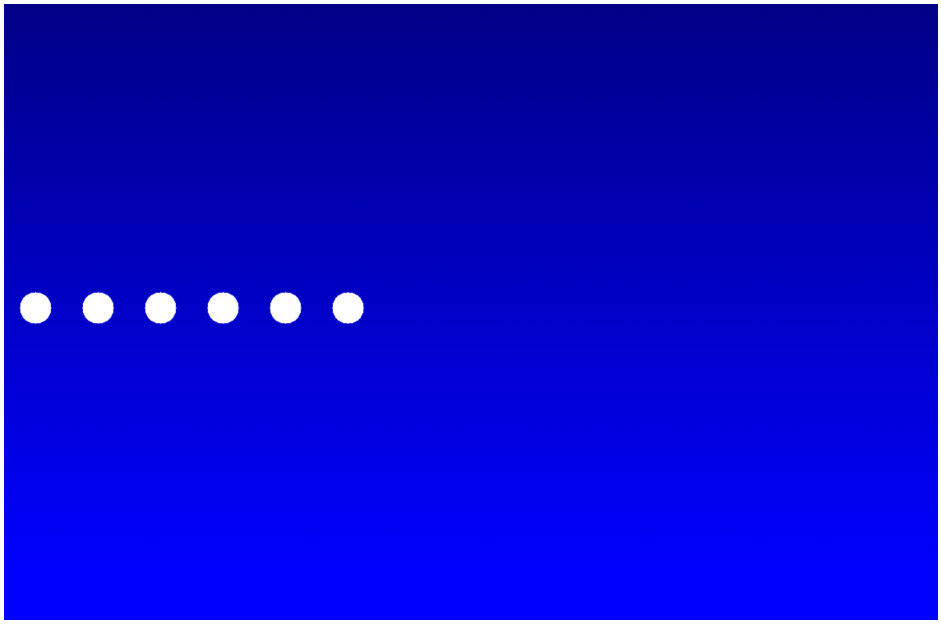
터미널 창에 ‘make’를 입력하면 아래와 같은 화면이 출력되고 CP2_2023_FALL_소원_TERM01.cpp가 실행된다. 그런데 make만 입력하면 코드를 실행하기만 할 뿐 output.png가 만들어지지는 않는다. 그래서 ‘make output.png’를 입력하면 png파일도 만들어진다.

```
manymogi@manymogi:~/term_project$ make
g++ -std=c++11 -o CP2_2023_FALL_소원_TERM01 CP2_2023_FALL_소원_TERM01.o -lpng
manymogi@manymogi:~/term_project$ make output.png
./CP2_2023_FALL_소원_TERM01
홀수입니까? 짝수입니까? (홀수면 1, 짝수면 2를 입력하시오): 
```

파일이 만들어지고 기존의 파일을 지우고 다시 컴파일을 하고 싶다면
‘make clean’명령어를 사용하면 된다. 그 후 다시 ‘make’ 또는 ‘make output.png’명령어를 입력하면 된다.

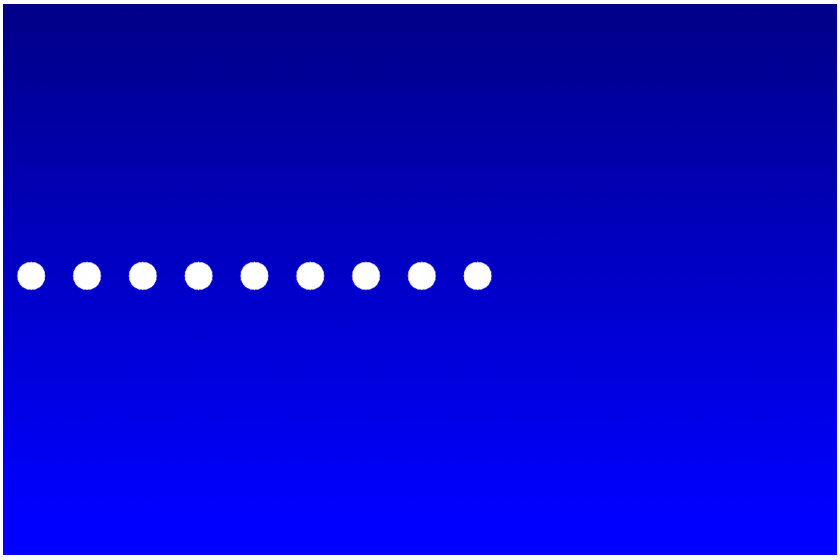
4.2 결과 출력

make output.png 명령어를 입력해 실행했다고 하자.
기본 출력 화면은 아래와 같다.



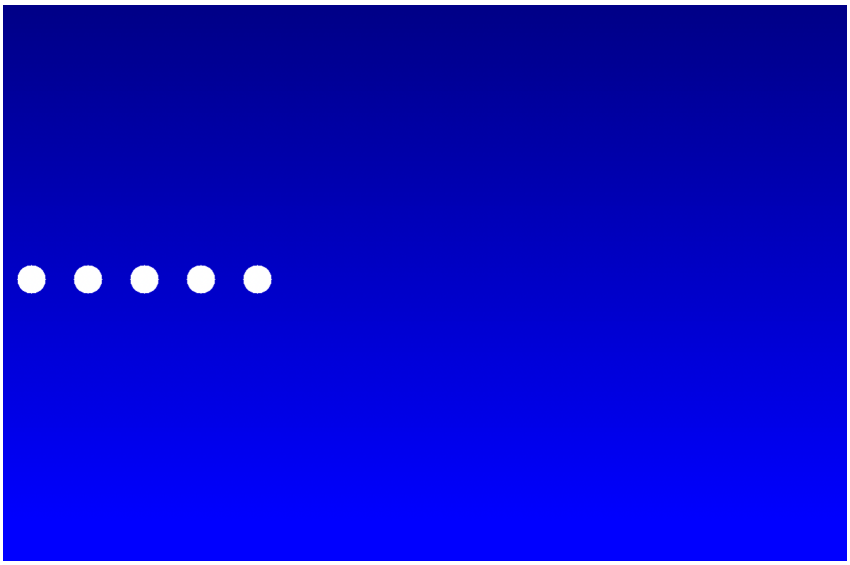
```
홀수입니까? 짝수입니까? (홀수면 1, 짝수면 2를 입력하시오): 1
이겼습니다!(현재 구슬 수: 9)
```

플레이어는 홀수냐 짝수냐를 묻는 질문에 1을 입력하고 이겼다. 입력 결과 구슬이 9개가 되었으니 컴퓨터가 출력한 랜덤 값은 3일 것이다. 홀짝을 맞혀 구슬 3개를 얻은 출력 화면은 아래와 같다.



```
구슬 몇 개를 걸겠습니까? (현재 구슬 수: 9): 4
졌습니다!(현재 구슬 수: 5)
```

이번에는 플레이어가 구슬을 걸 차례이다. 플레이어는 4(짝수)를 걸었고, 컴퓨터를 이를 맞혔다. 따라서 플레이어가 건 구슬의 수만큼 잃는다. 남은 구슬 수 5개의 출력 화면은 아래와 같다.



```
홀수입니까? 짝수입니까? (홀수면 1, 짝수면 2를 입력하시오): 1
당신은 모든 구슬을 얻었습니다!
```

다시 플레이어가 홀수, 짝수를 맞추는 차례가 왔다. 플레이어는 1을 입력했고 승리했다. 결과적으로 모든 구슬을 얻었다. 이 경우 WIN 화면이 출력된다.

위와 같은 방법으로 플레이어가 구슬을 모두 잃는다면 LOSE 화면을 출력한다. LOSE 화면 출력 값은 아래와 같다.

제 5장 결론

홀짝 게임을 구상하기 전 간단한 도형 이미지를 출력하는 예제를 만들었는데, 여러 번 실행을 하는 예제의 경우 예를 들면 출력하는 화면의 크기가 점점 작아지는 코드의 경우 기존의 파일에 새로 갱신되는 것이 아니라 out_1, out_2 같은 식으로 파일이 계속 생겼다. 이 부분을 보완하고자 홀짝 게임을 설계할 때는 png 파일이 계속 새로 생기는 것이 아니라 기존의 파일에 출력값만 계속 갱신할 수 있게끔 하였다. 파일을 단순히 위에 덮어쓰고 끝이 아니라 updateImage 함수가 실행되는 과정에서 발생할 수 있는 다양한 오류들을 감안해야 한다는 점이 특히 어려웠다. 그럼에도 원하는 결과가 나와 만족스럽다.

게임이 끝나면 WIN 또는 LOSE를 출력하는 부분도 많은 시행착오를 겪었다. 대략적인 좌표를 짐작하기 어려울 뿐만 아니라 글자를 출력하면 계속 화면 밖으로 나가버리거나 옆으로 쏠렸기 때문이다. 결과적으로는 알아볼 수 있을 정도의 글자가 출력되었지만 이를 위하여 픽셀 값을 10단위에서 100단위까지 일일이 조절한 점이 아쉬웠다. 픽셀 값을 섬세하게 조절하기보다는 화면 크기가 저장된 변수만을 이용해 글자를 출력했다면 더 좋았을 것 같다.

화면에 출력된 글자가 너무 얇아 잘 보이지 않는다는 단점이 있다. 이를 보완하고자 처음에는 drawLine 함수에 선의 두께를 입력하는 변수를 만들어 더 두꺼운 글자도 출력할 수 있게 하려 했는데 대각선으로 나아가는 선의 경우 선의 마무리가 깔끔하지 않아 결국 기존의 얇은 글자를 출력했다. 시간이 더 있었다면 이 부분을 개선하고 싶다.

*참고문헌

1. https://modoocode.com/311